



Frontlines Edutech

SQL Learning Guide

Set Operations | Grouping & Sorting | Subqueries | CTE

#	Topic	Subtopics
1	Set Operations	UNION, UNION ALL, INTERSECT, EXCEPT
2	Grouping & Sorting	DISTINCT, ORDER BY, GROUP BY, HAVING
3	Subqueries	Nested query, Correlated Subquery
4	Common Table Expressions	CTE syntax, chained CTEs, CTE vs Subquery
5	Interview Q & A	15 real-world fresher questions
6	Practice Questions	20 exercises on two sample tables

<https://www.frontlinesedutech.com>



1. Set Operations

Set operations let you combine the results of two or more SELECT queries into one result set. Each SELECT must return the same number of columns with compatible data types.

Quick Tip: All set operations: same column count + compatible data types in each SELECT.

1.1 UNION

Definition

UNION combines results from two SELECT queries and automatically REMOVES duplicate rows.
Only unique rows appear in the final result — like merging two lists and crossing out repeats.

Syntax

```
SELECT column1, column2 FROM table_one  
UNION  
SELECT column1, column2 FROM table_two;
```

Example — Input Tables

Two branch tables store student cities. We want one unique list.

Table: flm_branch_hyd (Input 1)

student_id	city
S01	Hyderabad
S02	Mumbai
S03	Hyderabad

Table: flm_branch_del (Input 2)

student_id	city
S04	Delhi
S05	Mumbai
S06	Bangalore

Query

```
SELECT city FROM flm_branch_hyd  
UNION  
SELECT city FROM flm_branch_del;
```



Output

Result (duplicates removed — Mumbai appears once)

city
Hyderabad
Mumbai
Delhi
Bangalore

Rules for UNION

- Both SELECT statements must return the SAME number of columns.
- Corresponding columns must have COMPATIBLE data types (INT with INT, etc.).
- UNION removes duplicates automatically — like DISTINCT on the merged set.
- Column names in the result come from the FIRST SELECT statement.
- ORDER BY can appear only ONCE, at the very end of the combined query.

1.2 UNION ALL

Definition

UNION ALL combines results from two queries but KEEPS ALL rows — including duplicates.
It is faster than UNION because it skips the duplicate-removal step.

Syntax

```
SELECT column1 FROM table_one
UNION ALL
SELECT column1 FROM table_two;
```

Example — Same Input Tables as UNION above

```
SELECT city FROM flm_branch_hyd
UNION ALL
SELECT city FROM flm_branch_del;
```



Output

Result (all 6 rows kept — Mumbai appears TWICE)

city
Hyderabad
Mumbai
Hyderabad
Delhi
Mumbai
Bangalore

Quick Tip: Use UNION ALL when you need all records (e.g., audit logs) or when you are sure there are no duplicates — it is significantly faster than UNION.

1.3 INTERSECT

Definition

INTERSECT returns only the rows that exist in BOTH query results — the overlap. Think of two circles in a Venn diagram: INTERSECT is the middle part.

Syntax

```
SELECT column1 FROM table_one
INTERSECT
SELECT column1 FROM table_two;
```

Example

Which cities appear in BOTH branch tables?

```
SELECT city FROM flm_branch_hyd
INTERSECT
SELECT city FROM flm_branch_del;
```



Output

Result (only cities present in BOTH tables)

city
Mumbai

Note:

INTERSECT is NOT available in MySQL.

In MySQL, achieve the same result with: `SELECT city FROM t1 WHERE city IN (SELECT city FROM t2);`

1.4 EXCEPT

Definition

EXCEPT returns rows from the FIRST query that do NOT appear in the SECOND query. It subtracts the second set from the first — like 'what is in A but not in B'. (Called MINUS in Oracle and some older versions of MySQL.)

Syntax

```
SELECT column1 FROM table_one
EXCEPT
SELECT column1 FROM table_two;
```

Example

Find cities that are in the Hyderabad branch but NOT in the Delhi branch:

```
SELECT city FROM flm_branch_hyd
EXCEPT
SELECT city FROM flm_branch_del;
```



Output

Result (cities in branch_hyd but NOT in branch_del)

city
Hyderabad

Set Operations — Side-by-Side Comparison

Operator	What it Returns	Removes Duplicates?	MySQL Support?
UNION	All rows from both queries	Yes	Yes
UNION ALL	All rows including repeats	No (fastest)	Yes
INTERSECT	Rows common to BOTH queries	Yes	No (use IN)
EXCEPT	Rows in first NOT in second	Yes	No (use NOT IN)

2. Grouping and Sorting

Grouping and sorting clauses let you organize, summarise, and filter your query results. These are essential for reports and data analysis.

2.1 DISTINCT

Definition

DISTINCT removes duplicate rows from a SELECT result so that only unique values (or unique combinations of values) are returned.

Syntax

```
-- Single column
SELECT DISTINCT column1 FROM table_name;

-- Multiple columns (unique combination of both)
SELECT DISTINCT column1, column2 FROM table_name;
```



Example — Input

Table: flmstudents

student_id	student_name	city
S01	Rahul	Hyderabad
S02	Priya	Mumbai
S03	Amit	Hyderabad
S04	Sneha	Delhi
S05	Ravi	Mumbai

```
SELECT DISTINCT city FROM flmstudents;
```

Output

Result — unique cities only

city
Hyderabad
Mumbai
Delhi

Quick Tip: DISTINCT applies to ALL selected columns together, not just the first one. SELECT DISTINCT city, department returns unique (city, department) pairs.

2.2 ORDER BY

Definition

ORDER BY sorts query results by one or more columns.
ASC = ascending (A→Z, 1→100) — this is the default.
DESC = descending (Z→A, 100→1).
ORDER BY is always the LAST clause in a SELECT statement.



Syntax

```
SELECT col1, col2
FROM   table_name
ORDER BY col1 ASC;           -- or DESC

-- Multiple columns: primary sort, then secondary sort
ORDER BY col1 ASC, col2 DESC;
```

Example — Input

Table: flmstudents

student_id	student_name	marks
S01	Rahul	88
S02	Priya	72
S03	Amit	95
S04	Sneha	72
S05	Ravi	60

```
-- Sort by marks descending (highest first),
-- then by name ascending as tiebreaker
SELECT student_id, student_name, marks
FROM   flmstudents
ORDER BY marks DESC, student_name ASC;
```

Output

Result — sorted by marks DESC, then name ASC

student_id	student_name	marks
S03	Amit	95
S01	Rahul	88
S04	Sneha	72
S02	Priya	72
S05	Ravi	60



2.3 GROUP BY

Definition

GROUP BY groups rows that share the same value in one or more columns, then lets you run aggregate functions on each group.

Common aggregate functions used with GROUP BY:

COUNT(*) — count rows in the group

SUM(col) — add up values

AVG(col) — calculate average

MAX(col) — largest value

MIN(col) — smallest value

Syntax

```
SELECT    column1, aggregate_function(column2)
FROM      table_name
WHERE     condition                -- optional: filters rows BEFORE grouping
GROUP BY column1
ORDER BY column1;                 -- optional
```

Example — Input

Table: flmenrollments

enroll_id	student_name	course_id	marks
E01	Rahul	C101	88
E02	Priya	C102	72
E03	Amit	C101	95
E04	Sneha	C103	80
E05	Ravi	C102	60
E06	Kiran	C101	77

```
-- Count students and find average marks per course
SELECT    course_id,
          COUNT(*)      AS total_students,
          AVG(marks)    AS avg_marks,
          MAX(marks)    AS top_marks
FROM      flmenrollments
GROUP BY course_id
```



```
ORDER BY total_students DESC;
```

Output

Result — summary per course

course_id	total_students	avg_marks	top_marks
C101	3	86.67	95
C102	2	66.00	72
C103	1	80.00	80

Rules for GROUP BY

- Every column in SELECT must either appear in GROUP BY or be inside an aggregate function.
- WHERE filters rows BEFORE grouping; HAVING filters AFTER grouping.
- You can GROUP BY multiple columns: GROUP BY city, department.
- NULL values are all placed into one group together.

2.4 HAVING

Definition

HAVING filters groups created by GROUP BY — it is like WHERE but for aggregated data. You CANNOT use aggregate functions (SUM, COUNT, AVG...) inside a WHERE clause. You MUST use HAVING for that.

Syntax

```
SELECT    column1, aggregate_function(column2)
FROM      table_name
GROUP BY column1
HAVING    aggregate_function(column2) > some_value;
```

Example — same flmenrollments table

Show only courses with MORE THAN 1 student enrolled:



```
SELECT  course_id, COUNT(*) AS total_students
FROM    flmenrollments
GROUP BY course_id
HAVING  COUNT(*) > 1
ORDER BY total_students DESC;
```

Output

Result — courses with > 1 student

course_id	total_students
C101	3
C102	2

WHERE vs HAVING — Critical Difference

	WHERE	HAVING
When it filters	BEFORE grouping	AFTER grouping
Can use aggregates?	No — SUM/COUNT not allowed	Yes — SUM/COUNT allowed
Works on	Individual rows	Groups of rows
Example	WHERE marks > 60	HAVING COUNT(*) > 2

Quick Tip: Memory trick: WHERE = filter rows → then group → then HAVING = filter groups.

SQL Clause Execution Order (not writing order)

```
1. FROM      -- which table(s)?
2. WHERE     -- filter individual rows
3. GROUP BY  -- create groups
4. HAVING    -- filter groups
5. SELECT    -- choose columns / compute values
6. ORDER BY  -- sort final output
7. TOP/LIMIT -- limit number of rows
```



3. Subqueries

A subquery is a SELECT statement written INSIDE another SQL statement. The outer query uses the result of the inner (sub) query.

3.1 Simple (Non-Correlated) Subquery

Definition

A simple subquery runs ONCE, produces a result, and that result is handed to the outer query.

The inner query is fully independent — it does not reference any column from the outer query.

Syntax

```
SELECT column1
FROM   table_name
WHERE  column2 = (SELECT aggregate_function(column2)
                  FROM   table_name);

-- Subquery can also be used with IN, NOT IN, >, <, ANY, ALL
```

Example — Input

Table: flmstudents

student_id	student_name	marks	city
S01	Rahul	88	Hyderabad
S02	Priya	72	Mumbai
S03	Amit	95	Delhi
S04	Sneha	80	Mumbai
S05	Ravi	60	Bangalore

Find the student(s) with the HIGHEST marks:

```
-- Inner query runs first: finds MAX(marks) = 95
-- Outer query: WHERE marks = 95
SELECT student_name, marks
```



```
FROM flmstudents
WHERE marks = (SELECT MAX(marks) FROM flmstudents);
```

Output

Result

student_name	marks
Amit	95

Subquery using IN — Example

Find students enrolled in course 'C101' (use subquery instead of JOIN):

```
SELECT student_name
FROM flmstudents
WHERE student_id IN (SELECT student_id
                     FROM flmenrollments
                     WHERE course_id = 'C101');
```

Output

Result

student_name
Rahul
Amit
Kiran

Subquery in FROM clause — Derived Table

A subquery in the FROM clause creates a temporary table (called a derived table or inline view):

```
SELECT dept, avg_salary
FROM (
    SELECT department AS dept,
           AVG(salary) AS avg_salary
    FROM flmemployees
    GROUP BY department
) AS dept_summary
WHERE avg_salary > 50000;
```



3.2 Correlated Subquery

Definition

A correlated subquery REFERENCES a column from the OUTER query inside the inner query.

It runs ONCE FOR EVERY ROW processed by the outer query — making it more powerful but slower.

Use it for row-by-row comparisons where the inner result depends on each outer row.

Syntax

```
SELECT outer_col1, outer_col2
FROM   outer_table AS o
WHERE  outer_col3 operator (
        SELECT aggregate(inner_col)
        FROM   inner_table AS i
        WHERE  i.shared_col = o.shared_col  -- references outer alias 'o'!
      );
```

Example — Input

Table: flmememployees

emp_id	emp_name	department	salary
E01	Rahul	IT	75000
E02	Priya	HR	65000
E03	Amit	IT	55000
E04	Sneha	HR	70000
E05	Ravi	IT	80000

Find employees who earn MORE than their own department's average salary:

```
-- For each employee row (outer), inner calculates AVG for THAT department
SELECT emp_name, department, salary
FROM   flmememployees AS e
WHERE  salary > (
        SELECT AVG(salary)
        FROM   flmememployees AS inner_e
        WHERE  inner_e.department = e.department  -- correlated!
      );
```



Output

Result — above-average earners per department

emp_name	department	salary
Ravi	IT	80000
Sneha	HR	70000

Simple vs Correlated Subquery — Comparison

Feature	Simple Subquery	Correlated Subquery
Depends on outer query?	No — fully independent	Yes — references outer columns
Executes how many times?	Once only	Once per row of outer query
Performance	Faster	Slower (but more powerful)
Best used for	Single computed filter value	Row-by-row comparisons

4. Common Table Expressions (CTE)

Definition

A CTE is a temporary, named result set you define BEFORE the main query using the WITH keyword.

You can then use that name just like a table inside the rest of the query.

CTEs make complex, multi-step queries much easier to read and debug.

A CTE exists ONLY for the duration of the single query it belongs to.

4.1 Basic CTE — Syntax

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
    WHERE condition  
)  
SELECT * FROM cte_name;  
  
-- 'cte_name' can be used like a regular table in the main SELECT.
```



Example — Input

Table: flmstudents

student_id	student_name	marks	city
S01	Rahul	88	Hyderabad
S02	Priya	72	Mumbai
S03	Amit	95	Delhi
S04	Sneha	80	Mumbai
S05	Ravi	60	Bangalore

List all students who scored above 80, showing only their name and marks, sorted highest first:

```
WITH high_scorers AS (  
    SELECT student_name, marks  
    FROM flmstudents  
    WHERE marks > 80  
)  
SELECT student_name, marks  
FROM high_scorers  
ORDER BY marks DESC;
```

Output

Result

student_name	marks
Amit	95
Rahul	88



4.2 Multiple (Chained) CTEs

You can define several CTEs in one WITH block, separated by commas. Later CTEs can reference earlier ones.

```
WITH
  top_students AS (
    SELECT student_id, student_name, marks
    FROM   flmstudents
    WHERE  marks >= 80
  ),
  enrolled_in_c101 AS (
    SELECT student_id
    FROM   flmenrollments
    WHERE  course_id = 'C101'
  )
SELECT s.student_name, s.marks
FROM   top_students      AS s
JOIN   enrolled_in_c101 AS e ON s.student_id = e.student_id;
```

Output

Result — high scorers also in course C101

student_name	marks
Rahul	88
Amit	95

Quick Tip: In a chained CTE, the second CTE can reference the first CTE just like a table — no nesting required. This keeps the logic flat and readable.

4.3 CTE vs Subquery — When to Use Which

Feature	Subquery	CTE (WITH)
Readability	Gets hard with deep nesting	Clean and easy to follow
Reusability in same query	Must copy-paste the subquery	Reference the CTE name anywhere
Debugging	Hard to test individual steps	Easy — test each CTE separately
Performance	Same as CTE in most cases	Same as subquery in most cases
Syntax position	Inline inside the query	Separate WITH block at the top



Quick Tip: Prefer CTEs whenever a query has more than two levels of nesting — they are far easier to read, maintain, and share with teammates.

5. Real-Time Interview Questions & Answers

These questions are commonly asked in SQL Server interviews for fresher and junior developer roles. Study each carefully!

Set Operations

Q: What is the difference between UNION and UNION ALL?

A: UNION combines results of two queries and REMOVES duplicate rows — it applies an implicit DISTINCT. UNION ALL combines all rows including duplicates and is FASTER because it skips the de-duplication step. Use UNION when you need unique results; use UNION ALL when performance matters or duplicates are acceptable (e.g., combining transaction logs).

Q: Can you use ORDER BY with UNION? If yes, how?

A: Yes — but only ONE ORDER BY clause is allowed, placed at the very end of the entire combined query (after the last SELECT). It sorts the final merged result. You cannot put ORDER BY in the middle between individual SELECT statements.

Q: What is INTERSECT and when would you use it?

A: INTERSECT returns only the rows that exist in BOTH result sets — the common overlap. Use it when you need records that satisfy two independent conditions in separate queries. For example: customers who placed orders in January AND February. Note: INTERSECT is NOT available in MySQL; use INNER JOIN or WHERE col IN (subquery) instead.

Q: How is EXCEPT different from NOT IN?

A: Both exclude rows, but EXCEPT works on entire rows from two result sets and handles NULLs correctly (NULL = NULL in EXCEPT). NOT IN on a column with NULLs can return unexpected empty results because NULL != anything in SQL. EXCEPT is safer and more readable when excluding entire row sets.



Grouping & Sorting

Q: What is the difference between WHERE and HAVING?

A: WHERE filters individual rows BEFORE GROUP BY runs. HAVING filters groups AFTER GROUP BY runs. The key rule: you CANNOT use aggregate functions (SUM, COUNT, AVG, MAX, MIN) in a WHERE clause — you must use HAVING for that. Example: WHERE salary > 50000 filters rows; HAVING COUNT(*) > 3 filters groups.

Q: Can you use HAVING without GROUP BY?

A: Yes — technically HAVING without GROUP BY treats the entire result as one group. For example: SELECT COUNT(*) FROM flmstudents HAVING COUNT(*) > 10; returns the count only if more than 10 rows exist. It is valid but uncommon in practice.

Q: What is the difference between DISTINCT and GROUP BY?

A: Both can eliminate duplicates, but they serve different purposes. DISTINCT removes duplicate rows from the SELECT output — simple and straightforward. GROUP BY groups rows so you can apply aggregate functions (COUNT, SUM, etc.) on each group. Use DISTINCT when you only need unique values; use GROUP BY when you also need calculations per group.

Q: In what order does SQL Server execute the clauses of a SELECT statement?

A: The logical execution order is: (1) FROM, (2) WHERE, (3) GROUP BY, (4) HAVING, (5) SELECT, (6) ORDER BY, (7) TOP/OFFSET-FETCH. This is why you cannot reference a column alias from SELECT inside a WHERE clause — SELECT has not been processed yet at that point.

Subqueries

Q: What is a subquery? Where can it appear in a SQL statement?

A: A subquery is a SELECT statement nested inside another SQL statement. It can appear in: (1) WHERE clause — to filter using a computed value; (2) FROM clause — as a derived/inline table; (3) SELECT clause — to compute a per-row value; (4) HAVING clause — to filter groups with an aggregated value.

**Q: What is a correlated subquery? How does it differ from a regular subquery?**

A: A regular subquery runs ONCE, independently of the outer query. A correlated subquery references a column from the outer query, so it runs ONCE PER ROW of the outer query. This makes it more powerful for row-by-row comparisons (e.g., 'employees earning above their department's average') but also slower. Recognize it by the outer table alias appearing inside the inner query.

Common Table Expressions (CTE)**Q: What is a CTE and how is it different from a subquery?**

A: A CTE (Common Table Expression) is a temporary named result set defined using WITH at the top of a query. The key advantages over a subquery: (1) a CTE can be referenced MULTIPLE TIMES in the main query without repeating the code; (2) CTEs are easier to read and debug; (3) CTEs can reference each other (chaining). Both have similar performance in most cases.

Q: How many CTEs can you define in one WITH block?

A: There is no hard limit — you can define as many CTEs as needed in one WITH block, separated by commas. Each CTE can reference the CTEs defined before it in the same block. This chaining keeps complex multi-step logic flat and readable without deep nesting.



6. Practice Questions

All practice questions use the two tables defined below. Write your queries in SQL Server Management Studio (SSMS) and verify your output.

Reference Tables — Use These for All Practice Questions

Table 1: flmstudents (student information)

student_id	student_name	city	course_id	marks	is_active
S01	Rahul Sharma	Hyderabad	C101	88	1
S02	Priya Singh	Mumbai	C102	72	1
S03	Amit Kumar	Delhi	C101	95	1
S04	Sneha Reddy	Mumbai	C103	80	0
S05	Ravi Verma	Bangalore	C102	60	1
S06	Meena Das	Hyderabad	C101	77	1
S07	Arjun Nair	Delhi	C103	91	0
S08	Kavya Rao	Bangalore	C102	55	1
S09	Rohit Patel	Hyderabad	C104	68	1
S10	Divya Mehta	Mumbai	C104	74	0

Table 2: flmcourses (course information)

course_id	course_name	instructor	fees	city
C101	SQL Fundamentals	Ravi Kumar	5000	Hyderabad
C102	Python Basics	Priya Sharma	6000	Mumbai
C103	Data Analysis	Amit Verma	7500	Delhi
C104	Excel for Data	Sneha Roy	4000	Bangalore
C105	Power BI	Kiran Das	8000	Hyderabad

Create these tables in SSMS with the INSERT statements below:

```
CREATE TABLE flmstudents (student_id VARCHAR(5) PRIMARY KEY, student_name  
VARCHAR(100) NOT NULL,  
city NVARCHAR(100), course_id VARCHAR(5), marks INT CHECK(marks BETWEEN 0  
AND 100), is_active BIT DEFAULT 1);
```



```
CREATE TABLE flmcourses (course_id VARCHAR(5) PRIMARY KEY, course_name  
VARCHAR(100) NOT NULL,  
instructor VARCHAR(100), fees DECIMAL(8,2), city NVARCHAR(100));
```

DISTINCT & ORDER BY — Questions 1 to 4

- 1 Write a query to display all unique cities from the flmstudents table, sorted A to Z.
- 2 List all unique (city, course_id) combinations from flmstudents. Sort by city ascending, then course_id ascending.
- 3 Display all students sorted by marks DESCENDING. If two students have the same marks, sort by student_name ASCENDING.
- 4 Show the top 3 highest-scoring students (student_name, marks) using ORDER BY and TOP.

GROUP BY & HAVING — Questions 5 to 9

- 5 Count how many students are enrolled in each course_id. Show course_id and student_count.
- 6 Find the average, maximum, and minimum marks per city. Show city, avg_marks, max_marks, min_marks. Sort by avg_marks descending.
- 7 Show only those courses (course_id) that have MORE THAN 2 students enrolled in flmstudents.
- 8 Find the total fees per city from flmcourses. Only show cities where total fees exceed 10,000.
- 9 Count how many ACTIVE students (is_active = 1) are in each city. Show city and active_count. Exclude cities with fewer than 2 active students.



Set Operations — Questions 10 to 14

- 10** Use UNION to get a single list of all unique cities appearing in EITHER flmstudents OR flmcourses.
- 11** Use UNION ALL to combine all cities from flmstudents and flmcourses (including repeats). How many total rows do you get?
- 12** Use INTERSECT to find cities that appear in BOTH flmstudents and flmcourses.
- 13** Use EXCEPT to find cities that exist in flmstudents but do NOT exist in flmcourses.
- 14** CHALLENGE: Use UNION to list all course_ids from flmstudents together with all course_ids from flmcourses. Remove duplicates and sort the result.

Subqueries — Questions 15 to 17

- 15** Write a subquery to find all students whose marks are ABOVE the overall average marks in flmstudents.
- 16** Use a subquery with IN to find the names of students enrolled in courses that cost more than 6,000 fees (use flmcourses to find those course_ids).
- 17** CORRELATED: Write a correlated subquery to find students who scored HIGHER than the average marks of students in their own city.

CTE — Questions 18 to 20

- 18** Write a CTE called city_summary that calculates the average marks per city from flmstudents. Then SELECT all cities where the average marks > 75 from the CTE.
- 19** Create TWO CTEs: (1) active_students — all students where is_active = 1; (2) expensive_courses — all courses where fees >= 6000. Join them on course_id and display matching student_name, course_name, and fees.